

FFresco / Totem SaaS - Backend Decisions & DynamoDB Setup

1. Objetivo del documento

Este documento guarda las decisiones iniciales del backend para el proyecto de menú digital, tótem de autoatendimento, operaciones internas y administración.

La idea es que este archivo funcione como fuente de verdad para futuras conversaciones, implementación con Claude Code/Cursor y documentación del repositorio.

2. Decisión principal de arquitectura

Para el MVP se usará:

```
1 tabla DynamoDB genérica
1 API/Lambda customer-facing inicial
Separación lógica por bounded contexts
```

La separación lógica esperada a futuro será:

```
restaurant-api
catalog-api / menu-api
ordering-api
payment-api
```

Pero para avanzar rápido en el MVP, se puede comenzar con una Lambda inicial:

```
ffresco-customer-api
```

Responsabilidad inicial:

```
Menú público
Contexto de mesa
Productos visibles
Creación de pedidos
Cuenta de mesa
```

Cuenta individual
Llamados al mozo

3. Decisión sobre DynamoDB

Se usará una tabla inicial:

ffresco-core-dev

Con claves genéricas:

PK: pk
SK: sk

Y un índice secundario global inicial:

GSI1
GSI1PK: gsi1pk
GSI1SK: gsi1sk

Motivo:

- Permite single table design.
- Permite guardar distintos tipos de entidad en la misma tabla.
- Evita crear muchas tablas al comienzo.
- Permite búsquedas alternativas como resolver una mesa pública desde el QR.
- Mantiene bajo el costo inicial.

4. Bounded contexts previstos

4.1 Restaurant / Business Context

Responsable de:

Tenant
Restaurant
Branch / Sucursal
Tables / Mesas
QR codes

Kiosks / Tótems
Business settings
Currency
Language
Service fee
Opening hours

4.2 Catalog / Menu Context

Responsable de:

Categories
Products
Price lists
Public menu projection
Availability
Featured products
Promotions
Basic stock

4.3 Ordering Context

Responsable de:

Table sessions
Orders
Order items
Kitchen status
Bill
Close bill requests
Waiter calls

4.4 Payment Context

Responsable futuro de:

Payment intent
Cash payment
Card payment
Mercado Pago
Refunds

Payment status
Receipts

5. Concepto clave: tablePublicId

No usar como identificador público valores simples como:

mesa-01

Porque en un SaaS multi-tenant muchas sucursales pueden tener una mesa 1.

Para los QR públicos usar:

tablePublicId = tbl_pub_8H7K2X

Ejemplo de URL:

/menu/tbl_pub_8H7K2X

O endpoint:

GET /public/menu?tableId=tbl_pub_8H7K2X

Internamente ese ID resuelve:

tenantId
branchId
tableId interno
tableName
restaurantName
activeMenuId

6. Concepto clave: TableSession

Una mesa física no es lo mismo que una cuenta abierta.

Se usará el concepto:

TableSession

Ejemplo:

Mesa 140
Session actual: ts_20260515_001
Estado: OPEN

Esto permite diferenciar:

Mesa 140 al mediodía
Mesa 140 a la noche
Mesa 140 mañana

Los pedidos y llamados del mozo se guardan dentro de la sesión activa.

7. Modelo base de ítems DynamoDB

7.1 Tenant / Restaurant metadata

pk = TENANT#t001
sk = METADATA
entityType = TENANT

Atributos esperados:

```
{  
  "tenantId": "t001",  
  "restaurantName": "Pertinho do Ceu",  
  "logoUrl": "https://...",  
  "language": "pt-BR",  
  "currency": "BRL",  
  "serviceFeeRate": 0.10  
}
```

7.2 Branch / Sucursal

```
pk = TENANT#t001  
sk = BRANCH#b001  
entityType = BRANCH
```

Atributos esperados:

```
{  
  "branchId": "b001",  
  "name": "Sucursal Centro",  
  "address": "..."  
}
```

7.3 Table / Mesa

```
pk = TENANT#t001#BRANCH#b001  
sk = TABLE#table001  
entityType = TABLE  
gsi1pk = TABLE_PUBLIC#tbl_pub_8H7K2X  
gsi1sk = METADATA
```

Atributos esperados:

```
{  
  "tenantId": "t001",  
  "branchId": "b001",  
  "tableId": "table001",  
  "tablePublicId": "tbl_pub_8H7K2X",  
  "tableName": "Mesa 140",  
  "active": true,  
  "activeMenuId": "menu001"  
}
```

7.4 Public Menu Category

```
pk = PUBLIC_MENU#t001#b001#menu001  
sk = CATEGORY#001#cat-bebidas  
entityType = MENU_CATEGORY
```

Atributos esperados:

```
{  
  "categoryId": "cat-bebidas",  
  "name": "Bebidas",  
  "imageUrl": "https://...",  
  "order": 1  
}
```

7.5 Public Menu Product

```
pk = PUBLIC_MENU#t001#b001#menu001  
sk = PRODUCT#cat-bebidas#001#prod-coca-zero  
entityType = MENU_PRODUCT
```

Atributos esperados:

```
{  
  "productId": "prod-coca-zero",  
  "name": "Coca Cola Zero",  
  "description": "Lata 350ml gelada",  
  "price": 8.9,  
  "imageUrl": "https://...",  
  "categoryId": "cat-bebidas",  
  "featured": true,  
  "available": true  
}
```

7.6 Table Session

```
pk = TABLE_SESSION#ts001
sk = METADATA
entityType = TABLE_SESSION
```

Atributos esperados:

```
{
  "tableSessionId": "ts001",
  "tenantId": "t001",
  "branchId": "b001",
  "tableId": "table001",
  "tablePublicId": "tbl_pub_8H7K2X",
  "status": "OPEN",
  "openedAt": "2026-05-15T12:00:00Z"
}
```

7.7 Order

```
pk = TABLE_SESSION#ts001
sk = ORDER#2026-05-15T12:34:56Z#ord001
entityType = ORDER
gsi1pk = BRANCH#b001#ORDER_STATUS#PENDING
gsi1sk = 2026-05-15T12:34:56Z#ord001
```

Atributos esperados:

```
{
  "orderId": "ord001",
  "orderNumber": "1042",
  "customerName": "Fernando",
  "status": "PENDING",
  "subtotal": 17.8,
  "serviceFee": 1.78,
  "total": 19.58,
  "items": [
    {
      "productId": "prod-coca-zero",
      "name": "Coca Cola Zero",
      "quantity": 2,
      "unitPrice": 8.9,
```

```
    "notes": ""
  }
]
}
```

7.8 Waiter Call

```
pk = TABLE_SESSION#ts001
sk = WAITER_CALL#2026-05-15T12:40:00Z#wc001
entityType = WAITER_CALL
gsi1pk = BRANCH#b001#WAITER_CALL_STATUS#OPEN
gsi1sk = 2026-05-15T12:40:00Z#wc001
```

Atributos esperados:

```
{
  "waiterCallId": "wc001",
  "customerName": "Fernando",
  "phone": "+5581999991234",
  "reason": "CALL_WAITER",
  "status": "OPEN",
  "createdAt": "2026-05-15T12:40:00Z"
}
```

8. Endpoints públicos recomendados para el customer frontend

8.1 Public Menu

```
GET /public/menu?tableId={tablePublicId}
```

Devuelve contexto, categorías y productos en una sola respuesta.

Response esperada:

```
{
  "context": {
    "tableId": "tbl_pub_8H7K2X",
    "tableName": "Mesa 140",
    "restaurantName": "Pertinho do Ceu",
  },
}
```

```

    "restaurantLogoUrl": "https://...",
    "language": "pt-BR",
    "currency": "BRL",
    "serviceFeeRate": 0.1
  },
  "categories": [
    {
      "id": "cat-bebidas",
      "name": "Bebidas",
      "imageUrl": "https://...",
      "order": 1
    }
  ],
  "products": [
    {
      "id": "prod-coca-zero",
      "name": "Coca Cola Zero",
      "description": "Lata 350ml gelada",
      "price": 8.9,
      "imageUrl": "https://...",
      "categoryId": "cat-bebidas",
      "featured": true,
      "available": true
    }
  ]
}

```

8.2 Product Detail

```
GET /public/menu/products/{productId}?tableId={tablePublicId}
```

8.3 Place Order

```
POST /public/tables/{tablePublicId}/orders
```

Request:

```

{
  "customerName": "Fernando",
  "phone": "+5581999991234",
  "items": [

```

```
{
  "productId": "prod-coca-zero",
  "quantity": 2,
  "notes": ""
}
]
```

Regla importante:

El backend calcula precios y totales.
El frontend nunca debe enviar precios confiables.

8.4 My Orders

```
GET /public/tables/{tablePublicId}/orders?customerName={customerName}
```

8.5 Bill

```
GET /public/tables/{tablePublicId}/bill
```

8.6 Close Bill Request

```
POST /public/tables/{tablePublicId}/bill/close-request
```

Request:

```
{
  "customerName": "Fernando",
  "scope": "MINE"
}
```

Valores posibles:

MINE
TABLE

8.7 Waiter Call

`POST /public/tables/{tablePublicId}/waiter-call`

Request:

```
{
  "customerName": "Fernando",
  "phone": "+5581999991234",
  "reason": "CALL_WAITER"
}
```

Valores posibles:

CALL_WAITER
REQUEST_BILL
ASK_ORDER_STATUS
OTHER

9. Cómo resolver el flujo GET /public/menu

El endpoint hace:

1. Recibe tablePublicId.
2. Query en GSI1:
gsi1pk = TABLE_PUBLIC#{tablePublicId}
gsi1sk = METADATA
3. Obtiene tenantId, branchId, tableId y activeMenuId.
4. Query a la tabla principal:
pk = PUBLIC_MENU#{tenantId}#{branchId}#{activeMenuId}
5. Separa ítems entityType MENU_CATEGORY y MENU_PRODUCT.
6. Devuelve context + categories + products.

10. Cómo resolver el flujo POST /orders

El endpoint hace:

1. Recibe tablePublicId.
2. Resuelve la mesa usando GSI1.
3. Busca o crea TableSession OPEN.
4. Valida que los productos existan en el PUBLIC_MENU activo.
5. Calcula precios en backend.
6. Calcula subtotal, serviceFee y total.
7. Crea ORDER dentro de TABLE_SESSION.
8. Genera orderNumber.
9. Devuelve el pedido creado.

11. SAM / CloudFormation - DynamoDB Table

Agregar este recurso al `template.yaml`.

```
Resources:
  FfrescoCoreTable:
    Type: AWS::DynamoDB::Table
    Properties:
      TableName: !Sub "ffresco-core-${Environment}"
      BillingMode: PAY_PER_REQUEST
      AttributeDefinitions:
        - AttributeName: pk
          AttributeType: S
        - AttributeName: sk
          AttributeType: S
        - AttributeName: gsi1pk
          AttributeType: S
        - AttributeName: gsi1sk
          AttributeType: S
      KeySchema:
        - AttributeName: pk
          KeyType: HASH
        - AttributeName: sk
          KeyType: RANGE
      GlobalSecondaryIndexes:
        - IndexName: GSI1
          KeySchema:
            - AttributeName: gsi1pk
              KeyType: HASH
```

```
- AttributeName: gsi1sk
  KeyType: RANGE
  Projection:
    ProjectionType: ALL
  PointInTimeRecoverySpecification:
    PointInTimeRecoveryEnabled: true
  SSESpecification:
    SSEEnabled: true
  Tags:
    - Key: Project
      Value: ffresco
    - Key: Environment
      Value: !Ref Environment
```

12. SAM - Environment variable para la Lambda

Agregar a la Lambda:

```
Environment:
  Variables:
    FFRESCO_CORE_TABLE_NAME: !Ref FfrescoCoreTable
```

Si ya existe `Environment`, agregar solo la variable dentro de `Variables`.

13. SAM - Permisos DynamoDB para la Lambda

Agregar a la función Lambda:

```
Policies:
  - DynamoDBCrudPolicy:
      TableName: !Ref FfrescoCoreTable
```

Si se quiere ser más estricto, se puede reemplazar luego por una policy custom con acciones específicas:

```
dynamodb:GetItem
dynamodb:PutItem
dynamodb:UpdateItem
dynamodb:Query
dynamodb:BatchGetItem
```

Pero para MVP `DynamoDBCrudPolicy` es aceptable.

14. Pasos para crear la tabla vía SAM

Paso 1: agregar el recurso al template

Agregar `FfrescoCoreTable` dentro de `Resources`.

Paso 2: agregar variable de entorno a la Lambda

Agregar:

```
FFRESCO_CORE_TABLE_NAME: !Ref FfrescoCoreTable
```

Paso 3: agregar policy a la Lambda

Agregar:

```
Policies:
  - DynamoDBCrudPolicy:
      TableName: !Ref FfrescoCoreTable
```

Paso 4: validar template

```
sam validate --template-file template.yaml
```

Paso 5: build

```
sam build --template-file template.yaml
```

Paso 6: deploy

```
sam deploy
  --stack-name ffresco-customer-api-dev
  --capabilities CAPABILITY_IAM CAPABILITY_NAMED_IAM
  --region sa-east-1
  --no-confirm-changeset
  --no-fail-on-empty-changeset
  --parameter-overrides Environment=dev ReleaseVersion=local
```

Si usás bucket de artifacts explícito:

```
sam deploy
--stack-name ffresco-customer-api-dev
--s3-bucket <TU_BUCKET_SAM_ARTIFACTS>
--capabilities CAPABILITY_IAM CAPABILITY_NAMED_IAM
--region sa-east-1
--no-confirm-changeset
--no-fail-on-empty-changeset
--parameter-overrides Environment=dev ReleaseVersion=local
```

15. Notas de costo

Para laboratorio/MVP con:

```
DynamoDB On-Demand
1 GSI
Lambda
API Gateway HTTP API
S3 para frontend
CloudWatch Logs
```

El costo esperado con bajo tráfico debería ser muy bajo.

Estimación práctica:

```
USD 0 a USD 5 / mes
```

Probablemente más cerca de USD 0 a USD 1 si hay pocos requests, pocos datos y sin servicios caros.

Evitar por ahora:

```
NAT Gateway
RDS siempre prendido
OpenSearch
Provisioned Concurrency
ECS/Fargate continuo
EC2 permanente
```

16. Decisión final actual

```
Crear ffresco-core-${Environment}  
Usar pk/sk genéricos  
Crear GSI1 desde el inicio  
Modelar tablePublicId para QR  
Modelar TableSession para cuenta abierta  
Unificar endpoint del menú público en GET /public/menu  
Calcular precios siempre en backend  
Preparar separación futura en restaurant/catalog/ordering/payment APIs
```